Geospatial Data Analysis

Data Science with Applications in Natural Language Processing

Learning goals

By the end, we will be able to:

- **represent** points, lines and areas with GeoPandas
- **choose** a coordinate system and measure in metres
- **geocode** place names and handle ambiguity
- **clean** and **normalise** geographic text fields
- **map** diffusion and locality with interactive maps

Outline

1. What does geospatial mean?
2. Coordinate reference systems
3. Geocoding
4. Application: the atlas of Greek proverbs

One more layer: the “where”

What it can be

- points on a map
- administrative areas
- networks, routes
- mentions of places in text

What it brings along

- geocoding: not always accurate
- uneven coverage of areas
- the map as an exploration tool

When is an observation geospatial?

When it is tied to a place in space

Explicitly
coordinates
37.98, 23.73

Through an area
municipality, region
“Attica”

Indirectly
a mention in text
“...in Athens”

Three geometries



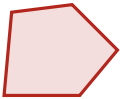
Point

city, sensor



Line

route, network



Polygon

municipality, region

The choice depends on the phenomenon, the scale and the question.

From a table to points

```
import pandas as pd, geopandas as gpd
from shapely.geometry import Point

data = pd.DataFrame({"id": [1, 2, 3],
                     "latitude":  [37.9838, 40.6401, 35.3387],
                     "longitude": [23.7275, 22.9444, 25.1442]})
geometry = [Point(xy) for xy in zip(data["longitude"], data["latitude"])]
gdf = gpd.GeoDataFrame(data, geometry=geometry, crs="EPSG:4326")
```

- Each row gets a geometry column
- Mind the order: Point(longitude, latitude), i.e. (x, y)

Points carry spatial meaning

```
athens       = Point(23.7275, 37.9838)
thessaloniki = Point(22.9444, 40.6401)
athens.distance(thessaloniki)     # 2.77
```

- Distance, proximity, containment: relations a plain table does not have
- But **2.77 of what?** Degrees, not kilometres

Lines

```
from shapely.geometry import LineString

route = LineString([(23.7348, 37.9755),
                    (23.7295, 37.9721),
                    (23.7209, 37.9764)])
route.length    # 0.016 (degrees)
```



- An ordered list of points: the **order** means direction or time

Polygons

```
from shapely.geometry import Polygon

area = Polygon([(23.70, 37.95), (23.78, 37.95), (23.83, 38.00),
                (23.78, 38.06), (23.70, 38.06), (23.67, 38.00),
                (23.70, 37.95)])          # closes at the first point

area.contains(athens)          # True
area.contains(route.centroid)  # True
```

- A closed line with extent: area, containment, overlap
- The centroid of a route: an **approximation** of where it is

Geometry and attributes

```
df = pd.DataFrame({"city": ["Athens", "Thessaloniki"],
                   "population": [3153000, 1100000]})
gdf = gpd.GeoDataFrame(df, crs="EPSG:4326", geometry=[
   Point(23.7275, 37.9838), Point(22.9444, 40.6401)])
```

attributes
what it is
city, population

geometry
where it is
geometry

A city: a point at national scale, a polygon at city scale.

Coordinate Reference System

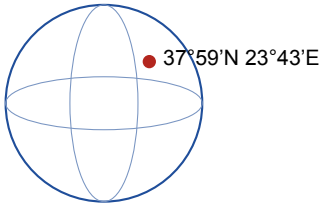
It defines how numbers map to places on Earth:

- the **shape** and size of the Earth
- the origin and orientation of the axes
- the **units** of measurement

EPSG:4326 (WGS 84)

- The most widely used system: GPS, apps, GIS
- Longitude and latitude in **degrees**
- Angles on a spheroid, not Cartesian coordinates

Good for storing and exchanging data.



From text to geometry

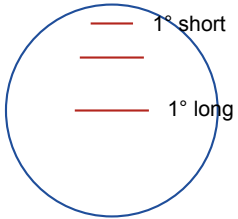
```
text_data = pd.DataFrame({
    "text": ["The event took place in Athens",
             "The conference was held in Thessaloniki",
             "The meeting took place in Patras"],
    "location": ["Athens", "Thessaloniki", "Patras"]})

geocode_dict = {"Athens":       Point(23.7275, 37.9838),
                "Thessaloniki": Point(22.9444, 40.6401),
                "Patras":       Point(21.7346, 38.2466)}
text_data["geometry"] = text_data["location"].map(geocode_dict)
gdf = gpd.GeoDataFrame(text_data, geometry="geometry", crs="EPSG:4326")
```

Before geometry, the data were not geospatial: the place was only text.

Why degrees are no good for measuring

- Distances come out in **degrees**
- Lengths and areas have no physical meaning
- A degree of longitude **shrinks** towards the poles



Projecting to metres: UTM 34N

```
gdf_projected = gdf.to_crs(epsg=32634)

gdf.geometry[0].distance(gdf.geometry[1])                 # 2.77
gdf_projected.geometry[0].distance(gdf_projected.geometry[1]) # 302569.82
```

- A projected system: units in **metres**
- For Greece, often UTM zone 34N (EPSG:32634)
- Athens–Thessaloniki: **302.57 km**

How accurate is it?

Method	Athens–Thessaloniki
straight line in degrees	2.77°
straight line in UTM 34N	302.57 km
geodesic (on the ellipsoid)	302.54 km

- Every projection **distorts** a little
- A small difference here; it grows away from the projection's zone

At city scale

	EPSG:4326	EPSG:32634
Syntagma–Thiseio	0.014°	1,226 m
route (3 points)	0.016°	1,494 m
polygon in Athens	0.013 sq. deg.	128.8 km²

Syntagma–Thiseio is 1,226 metres, about 1.2 km.

Pitfall: always check the CRS

```
gdf.crs              # <Geographic 2D CRS: EPSG:4326>
gdf.length           # UserWarning: Geometry is in a geographic CRS...
gdf.to_crs(epsg=32634).length    # metres
```

- Before lengths, areas and distances: **which CRS?**
- Two tables in **different** CRSs cannot be combined before a transformation

Explicit and implicit location

	Explicit	**Implicit**
Form	coordinates, geometries	place names, descriptions
Source	GPS, sensors, databases	texts, metadata
Uncertainty	from **measurement**	from **interpretation**
Needs	nothing	geocoding

Geocoding



Non-spatial data are **enriched** with geographic information

Ambiguous place names

Within a language	“Larissa”: the city or the prefecture?
Across languages	“Alexandria”: in which country?
Historical	“Constantinople” or “Istanbul”
Vague reference	“in northern Greece”, “in the wider area”

Geocoding is a problem of **interpretation**, not mere lookup.

Example: which Larissa?

```
examples = pd.DataFrame({
   "text": ["The meeting took place in Larissa",
            "The village of Larissa is in Crete"],
   "toponym": ["Larissa", "Larissa"]})
```

- Same place name, two different places
- **Context** decides:
  - geographic: neighbouring areas
  - linguistic: “prefecture”, “village”
  - metadata: where the text comes from

Gazetteer: many candidates

```
gazetteer = gpd.GeoDataFrame({
    "name":       ["Larissa",  "Larissa"],
    "region":     ["Thessaly", "Crete"],
    "type":       ["city",     "settlement"],
    "population": [144651,     200],
    "geometry":   [Point(22.4176, 39.6390), Point(25.0580, 35.2580)]},
    crs="EPSG:4326")

gazetteer[gazetteer["region"] == "Thessaly"]   # disambiguation
```

- First **all** candidates, then a choice based on context

Heuristics and their pitfalls

- With no context: pick by population or frequency
- The big city **almost always** wins
- Small places disappear systematically: **bias**

Context **manages** uncertainty, it does not remove it. Our assumptions must be documented and reproducible.

The atlas of Greek proverbs

- More than 100,000 variants, each with a place of recording
- The place is **free text**, not a geometry
- The study's finding: the most widespread proverbs come from the mainland, the most local ones from the islands

Diffusion or locality?

Pavlopoulos, Louridas & Filos, EMNLP 2024.

Loading

```
url = ("https://github.com/greek-proverb-atlas/proverbs.gr/"
       "raw/refs/heads/main/data/proverbs.csv")
proverbs = pd.read_csv(url, index_col=0)
proverbs.shape      # (134493, 4)
proverbs.columns    # ['text', 'place', 'uri', 'collector']
```

- One row = one recording of a proverb
- place: where it was recorded · collector: who recorded it

Duplicates?

```
proverbs.text.duplicated().sum()    # 24993
proverbs[proverbs.text == "Ουδέν καινόν υπό τον ήλιον"]
```

text	place	collector
Ουδέν καινόν υπό τον ήλιον	Αμοργός	Ιωαννίδης, Εμμανουήλ
Ουδέν καινόν υπό τον ήλιον	Λακωνία	Νεστορίδης, Κ.

- “Nothing new under the sun”: same text, **different place and collector**, two valid recordings
- These “duplicates” are exactly the **diffusion** we study

Duplicates across the whole record

```
proverbs.duplicated().sum()      # 14029
```

- Same text, place, URI and collector
- What counts as a “duplicate” **depends on the question**

Discussion: when would you drop them, and when keep them?

The most frequent “place”: none

```
proverbs["place"].value_counts().head()
```

Άδηλου τόπου	(unknown place)	9,029
Επτάνησος	(Ionian Islands)	8,283
Αμοργός	(Amorgos)	7,473
Αχαΐα, Πάτρα	(Achaea, Patras)	7,431
Κύπρος	(Cyprus)	6,019

- 9,029 recordings with no place, only **3,767** unique texts
- Names follow no single format: “Κρήτη”, “Νάξος, Απείρανθος”

Normalisation: the hierarchy of a place

```
places = pd.Series(["ΚρήτηΛασίθι,", "Κρήτη, Λασίθι",
                    "Κρήτη Λασίθι,", "Κρήτη , Λασίθι"])
places.str.replace(",", ">", regex=False)
places.str.replace(r"\s*,\s*", ">", regex=True)
```

plain replace	regular expression
Κρήτη>Λασίθι	Κρήτη>Λασίθι
Κρήτη> Λασίθι	Κρήτη>Λασίθι
Κρήτη >Λασίθι	Κρήτη>Λασίθι
Κρήτη > Λασίθι	Κρήτη>Λασίθι

The regular expression \s*,\s*



- \s: any whitespace character (space, tab, newline)
- *: zero or more times
- Together: “a comma, with any amount of whitespace around it”

It does not just fix: it imposes one **canonical form**. More under: regular expressions.

Choosing the scale

```
proverbs["place"] = proverbs.place.str.replace(r"\s*,\s*", ">",
                                    regex=True)
proverbs["places_hierarchy"] = proverbs.place.str.split(">")
proverbs["area"] = proverbs.places_hierarchy.str[0]   # the broadest

proverbs.place.nunique()   # 853
proverbs.area.nunique()    # 134
```

place	area
Κρήτη>Λασίθι>Κεντρί	Κρήτη
Σίφνος>Αρτεμώνας	Σίφνος

Why the broadest level?

- **Less ambiguity**: a small village may be missing from the gazetteer, or appear several times
- **Fewer requests**: 134 instead of 853
- **Less noise**: vague or historically shifting boundaries

The cost: we lose the detail within each area.

Geocoding with Nominatim

```
from geopy.geocoders import Nominatim
from tqdm import tqdm
import time

geolocator = Nominatim(user_agent="proverbs-geocoder")
geolocs = {}
for area in tqdm(proverbs["area"].dropna().unique()):
    try:
        loc = geolocator.geocode(f"{area}, Ελλάδα")
        geolocs[area] = {"latitude":  loc.latitude  if loc else None,
                         "longitude": loc.longitude if loc else None}
    except Exception:
        geolocs[area] = {"latitude": None, "longitude": None}
    time.sleep(1)    # respect the service's rate limit
```

Naming the country in the query reduces ambiguity. Store the results so you never ask twice.

Where geocoding fails

Cause	Example
another language or a historical name	Φθιώτιδα → Phthiotis, Βοιωτία → Boeotia
wrong scale	“Western Crete” → Chania
outside present-day Greece	Asia Minor, Pontus, Cyprus
non-administrative area	Thrace → Rhodope

The application needed 22 manual corrections.

Manual correction

```
need_fixing = {"Phthiotis": ["Φθιώτιδα"], "Boeotia": ["Βοιωτία"],
               "Chania": ["Δυτική Κρήτη"], "Rhodope": ["Θράκη"],
               "Izmir": ["Μικρά Ασία"], ...}

for query, (area,) in need_fixing.items():
    loc = geolocator.geocode(query)
    geolocs[area] = {"latitude": loc.latitude,
                     "longitude": loc.longitude}

geolocs_df = pd.DataFrame(geolocs).T       # after the corrections!
```

Build the table **after** the corrections; otherwise it will not contain them.

In the published data, too

```
bal = pd.read_csv(".../proverbs.gr/main/data/balanced_corpus.csv")
bal.groupby("area")[["lat", "lon"]].first()
```

Area	Where the coordinate lands
Epirus	Kypseli, Athens
Arcadia	Arkadias Street, Petroupoli (Athens)
Asia Minor	Pyli, Boeotia
Aetolia	Syros
Pontus	Sarti, Chalkidiki

- The geocoder found **streets and villages** with the same name
- Even this lecture's fix is missing: Asia Minor $\rightarrow$ Izmir

Checked by reverse geocoding (Nominatim, reverse).

Coordinates in the original table

```
proverbs = proverbs.join(geolocs_df, on="area")
proverbs[["text", "place", "area", "latitude", "longitude"]].sample()
```

- From free text to **coordinates**, via the area
- Each proverb gets the location of its **area**, not its village

Diffusion: which proverbs travel?

```
named = proverbs[proverbs.collector != "anonymous"]   # 119493
located = named[named.latitude.notna()]

areas = located.groupby("text")["area"].agg(set)
areas = areas.map(lambda s: s - {"Άδηλου τόπου"})
multi = areas[areas.map(len) > 1]                     # multi-area
```

- Only **named** collectors: more reliable data
- A proverb is “multi-area” if recorded in **more than one** area

Before the location filter: 3,204 multi-area proverbs from named collectors.

A diffusion map with Folium

```
import folium

m = folium.Map(location=[39.07, 21.82], zoom_start=6)
for areas_of_proverb in multi:
    locs = [(geolocs[a]["latitude"], geolocs[a]["longitude"])
            for a in areas_of_proverb]
    folium.PolyLine(locs, color="red", weight=1,
                    opacity=0.1).add_to(m)
m.save("diffusion.html")
```

- Folium wants **(latitude, longitude)**; Shapely **(longitude, latitude)**
- Low opacity: patterns show without clutter

What the map shows

The map as a tool for exploration, not just presentation

- Areas with many lines: diffusion **hubs**
- Or simply areas with **many recordings**?
- The order of points here does not mean direction

Locality: single-place proverbs

```
proverbs_sp = named[~named.text.duplicated(keep=False)]
proverbs_sp = proverbs_sp[proverbs_sp.place != "Άδηλου τόπου"]
freq_places = proverbs_sp.place.value_counts().head(20)
```

Επτάνησος	(Ionian Islands)	8,041
Αμοργός	(Amorgos)	7,028
Αχαΐα>Πάτρα	(Achaea>Patras)	6,749

- keep=False: drop **every** occurrence of a repeated proverb

A locality map

```
m_sp = folium.Map(location=[39.07, 21.82], zoom_start=5)
for place, freq in freq_places.head(10).items():
    area = place.split(">")[0]
    if area not in geolocs:
        continue
    folium.CircleMarker(
        location=(geolocs[area]["latitude"], geolocs[area]["longitude"]),
        radius=freq / 500,
        popup=f"{area}: {freq} single-place proverbs",
        fill=True).add_to(m_sp)
```

Radius proportional to the count; the popup gives the exact number.

Pitfall: locality or collector?

Place	recordings	share of one collector
Achaea>Patras	7,431	99.9%
Amorgos	7,473	99.5%
Laconia	5,187	95.1%
Ionian Islands	8,283	83.9%
Cyprus	6,019	60.7%

- Many “single-place” proverbs where **one** collector worked hard
- The map may show **recording effort**, not culture

A combined map

```
m = folium.Map(location=[39.07, 21.82], zoom_start=5,
               tiles="cartodbpositron")
lines   = folium.FeatureGroup(name="Multi-area proverbs")
circles = folium.FeatureGroup(name="Single-place proverbs")
# ... PolyLine into lines, CircleMarker into circles ...
lines.add_to(m); circles.add_to(m)
folium.LayerControl().add_to(m)
```

- Red: diffusion · blue: locality
- A neutral background, layers that can be toggled

Summary

1. Geometry (where) and attributes (what) in one table
2. Degrees for storage, metres for measurement: always check the CRS
3. Geocoding is interpretation: context, scale, corrections
4. What counts as a duplicate depends on the question
5. A map also shows how the data were collected

Exercise 1: from a table to a GeoDataFrame

In balanced_corpus.csv (23 areas, 500 proverbs each):

- Take one coordinate per area and build a GeoDataFrame
- How many km is “Epirus” from “Ioannina”?

Remember: distances are measured in metres, not degrees.

Exercise 1: solution

```
coords = bal.groupby("area")[["lat", "lon"]].first()
g = gpd.GeoDataFrame(coords, crs="EPSG:4326",
      geometry=gpd.points_from_xy(coords.lon, coords.lat))
u = g.to_crs("EPSG:32634")                # UTM 34N, metres
u.loc["Ηπειρος"].geometry.distance(u.loc["Ιωάννινα"].geometry)
# 311 km
```

- points_from_xy(x=lon, y=lat): **longitude** first
- 311 km: “Epirus” is in Athens, 7 km from “Arcadia”

Exercise 2: points out of bounds

- With a rectangle around Greece (longitude 19.3–29.7, latitude 34.7–41.8), find the areas that fall **outside**
- Does the check catch the errors of the previous exercise?

Hint: shapely.geometry.box and GeoSeries.within.

Exercise 2: solution

```
from shapely.geometry import box
greece = box(19.3, 34.7, 29.7, 41.8)    # minx, miny, maxx, maxy
g[~g.within(greece)].index              # Κύπρος[''] (Cyprus)
```

- Cyprus is outside, but a **correct** record
- Epirus in Athens is **inside**: the check misses it

$\Rightarrow$ Bounds catch gross errors; “plausible” ones need a comparison with what we **expect**

Exercise 3: locality or collector?

- In the same file, for each area compute:
  - how many distinct collectors
  - the share of the most prolific one
- Which areas rest on a single person?

Exercise 3: solution

```
bal.groupby("area")["collector"].agg(
   n="nunique",
   top=lambda s: s.value_counts(normalize=True).iloc[0])
```

Area	collectors	share of the first
East Thrace	1	100%
Skyros	3	99.6%
Amorgos	4	97.6%
…		
Thrace	31	14.6%

- Balanced across areas $\neq$ balanced across **sources**

For discussion

1. The geocoder always picks the biggest city with a given name. Which places vanish from the map?
2. How would you separate cultural locality from the effect of one prolific collector?
3. You keep only the top level of each place. Which question can you no longer answer?